ISEP INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

Estudo e Implementação de uma Estufa Inteligente de Pequena Dimensão

Eduardo Miguel Moreira Junqueira e Pedro Miguel Ferreira Pinto

Mestrado em Engenharia Electrotécnica e de Computadores
Área de Especialização em Automação e Sistemas

Orientador: André Vaz da Silva Fidalgo

Universidade Técnica do Porto
Instituto Superior de Engenharia do Porto
Departamento de Engenharia Eletrotécnica

Novembro, 2026

Estudo e Implementação de uma Estufa Inteligente de Pequena Dimensão

Dissertação submetida por **Eduardo Miguel Moreira Junqueira e Pedro Miguel Ferreira Pinto** (estudante nº 1251561 e 1200919) para obtenção do Grau de Mestre em Engenharia Electrotécnica e de Computadores, Área de Especialização em Automação e Sistemas.

Orientador:

André Vaz da Silva Fidalgo, `anf@isep.ipp.pt`

Co-Orientador:

António José Matos de Meireles, `aem@isep.ipp.pt`

Júri:

Presidente:

Maria Isabel de Castro Lopes Martins Pinto Ferreira, DEE/ISEP

Vogais:

Nome do vogal, DDD/PPPP

Nome do vogal, DEE/ISEP

ISEP INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

Rua Dr. António Bernardino de Almeida 431, 4249–015 Porto

Novembro, 2026

ISEP INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

Declaração de Integridade

Eu, Eduardo Miguel Moreira Junqueira e Pedro Miguel Ferreira Pinto, estudante n.º 1251561 e 1200919, declaro ter conduzido este trabalho académico com integridade. Não plagiei ou apliquei qualquer forma de uso indevido de informações ou falsificação de resultados ao longo do processo que levou à sua elaboração.

Declaro que o trabalho apresentado neste documento é original e de minha autoria, não tendo sido utilizado anteriormente para nenhum outro fim. Adicionalmente declaro que:

☒ não utilizei quaisquer ferramentas de inteligência artificial generativa na elaboração desta dissertação;

☐ utilizei ferramentas de inteligência artificial generativa na elaboração desta dissertação, conforme descrito no Anexo "Declaração de Uso de Inteligência Artificial Generativa", do qual assumo plena responsabilidade.

Declaro ainda que tenho pleno conhecimento do Código de Conduta Ética da UTP.

ISEP, Porto, 11 de Novembro de 2026

Resumo

No âmbito do *software*, propõe-se o desenvolvimento de uma aplicação móvel para o sistema *Agriculture Internet of Things* (AgrIoT), que permitirá ao utilizador monitorizar e controlar remotamente a estufa através de uma *User Interface* (UI) simples e acessível. A aplicação comunicará com um servidor através de uma *Application Programming Interface* (API) sobre *HyperText Transfer Protocol* (HTTP), com troca de dados em formato *JavaScript Object Notation* (JSON), de modo a integrar os valores medidos pelos sensores e os comandos de atuação do sistema *Internet of Things* (IoT). Neste documento são apresentados o projeto da arquitetura de *software*, os módulos que a compõem, os procedimentos e vetores de teste previstos e a respetiva análise de custos.

Palavras-Chave: *Internet of Things*, Aplicação *Android*, Monitorização e Controlo, *layout*.

Abstract

Regarding the software, the development of a mobile application for the *Agriculture Internet of Things* (AgrIoT) system is proposed, allowing the user to remotely monitor and control the greenhouse through a simple and accessible *User Interface* (UI). The application will communicate with a server through an *Application Programming Interface* (API) over *HyperText Transfer Protocol* (HTTP), exchanging data in *JavaScript Object Notation* (JSON) format, in order to integrate the values measured by the sensors and the actuation commands of the IoT system. This document presents the software architecture design, the modules that compose it, the planned test procedures and vectors, and the corresponding cost analysis.

Keywords: Internet of Things, Android Application, Monitoring and Control, *layout*.

Índice

Lista de Figuras **ix**

Lista de Tabelas **xi**

Lista de Acrónimos **xiii**

1 Software **1**

1.1 Linguagens e Tecnologias 1

1.1.1 Aplicação *Android* 1

1.1.2 Servidor 2

1.1.3 Comunicação 2

1.1.4 Base de Dados 3

1.1.5 Resumo 4

1.2 Arquitetura de Software 5

1.2.1 Fluxo de Dados 5

1.2.2 Visão Geral da Arquitetura 5

1.3 Conceção da Interface (*Mockups*) 8

1.3.1 Visão Geral dos Ecrãs 8

1.3.2 *Design layout* 9

1.3.3 Utilizador 9

1.3.4 Perfil de Administrador 10

1.4 Trabalho Futuro 11

Referências **13**

Lista de Figuras

1.1 *Android developers use Kotlin* [1]. 2
1.2 Logótipo, *framework Flask* [7]. 2
1.3 Redirecionamento em HTTP [8]. 3
1.4 Arquitetura de transporte de dados *JavaScript Object Notation* (JSON) [9]. 3
1.5 Logótipo *SQLite database engine* [10]. 4
1.6 Arquitetura de *software* do *Agriculture Internet of Things* (AgrIoT) [13]. 6
1.7 Conjunto dos *mockups* da aplicação *android* AgrIoT [4]. 8

Lista de Tabelas

1.1 Resumo das linguagens e tecnologias utilizadas. 4
1.2 Ciclo de comunicação típico 5
1.3 Linguagem visual da interface da aplicação. 9
1.4 *Layouts* do administrador e operação *HyperText Transfer Protocol* (HTTP) associada. 10

Lista de Acrónimos

AgrIoT	*Agriculture Internet of Things*
API	*Application Programming Interface*
HTTP	*HyperText Transfer Protocol*
IDE	*Integrated Development Environment*
IoT	*Internet of Things*
JSON	*JavaScript Object Notation*
LAMEC	Laboratório de Mecatrónica
ML	Machine Learning
PID	Proporcional-Integral-Derivativo
REST	*Representational State Transfer*
SDK	*Software Development Kit*
SGBD	Sistema de Gestão de Bases de Dados
SQL	*Structured Query Language*
TCP	*Transmission Control Protocol*
UC	Unidade Curricular
UI	*User Interface*
URL	*Uniform Resource Locator*
VPN	*Virtual Private Network*

Capítulo 1

Software

Este capítulo descreve a componente de *software* do sistema *Internet of Things* (IoT) desenvolvido para a monitorização e o controlo da estufa inteligente designado de AgrIoT. O *software* constitui a camada responsável por recolher os dados medidos pelos sensores, armazená-los, processá-los e disponibilizá-los ao utilizador, bem como enviar os comandos de atuação ao *hardware*.

1.1 Linguagens e Tecnologias

Esta secção define as linguagens, *frameworks*, *design* e protocolos utilizados no desenvolvimento do *software*, de forma a enquadrar as decisões posteriores de arquitetura apresentadas na secção 1.2.

1.1.1 Aplicação *Android*

O *Android* é um sistema operativo *open-source*, baseado no núcleo (*kernel*) *linux*, concebido para dispositivos móveis, como telemóveis e *tablets*, entre outros, e constitui a principal alternativa ao sistema operativo *IOS* da *Apple*. O respetivo *Software Development Kit* (SDK) disponibiliza as bibliotecas, as ferramentas de compilação e os emuladores necessários ao desenvolvimento de aplicações para esta plataforma [1]. A aplicação será desenvolvida em *Kotlin*, uma linguagem de programação criada pela *JetBrains* e e adotada pela *Google I/O* em 2019 com suporte a *Android*, na figura seguinte 1.1 [2].



Figura 1.1: *Android developers use Kotlin* [1].

Como ambiente de desenvolvimento, será utilizado o *Android Studio*, na versão *Rabbit 2*, que é o *Integrated Development Environment* (IDE) oficial para esta plataforma. Este ambiente integra o editor de código, o editor visual de *layouts*, o emulador de dispositivos e as ferramentas de compilação [3].

Antes da implementação da aplicação, foram desenvolvidos *mockups* dos *layouts*, isto é, representações visuais da interface que permitem definir o aspeto e a organização dos elementos antes de escrever código. Para esse efeito será utilizado a ferramenta *web Figma* de *design* de interfaces, que permite conceber e partilhar a *User Interface* (UI) de aplicações *web*, móveis entre outras [4].

Entre as principais funcionalidades do *Figma* destacam-se a criação de *layouts* (*frames*) estáticos, com dimensões de dispositivos reais, a definição de componentes reutilizáveis, como botões e campos de texto, e a prototipagem interativa, que permite simular a navegação entre ecrãs sem escrever código [4].

1.1.2 Servidor

O *Python* é uma linguagem de programação de alto nível, com um aglomerado conjunto de bibliotecas amplamente usada em aplicações da *web*, desenvolvimento de *software*, ciência de dados e Machine Learning (ML) [5],[6]. O *Flask* é uma *framework* para o desenvolvimento de aplicações *web* e móvel em *Python*, caracterizada pela sua simplicidade e por permitir definir rotas que respondem a pedidos HTTP, representada em baixo na figura 1.2 [7].



Figura 1.2: Logótipo, *framework Flask* [7].

Neste projeto, o *Flask* é utilizado para implementar a *Application Programming Interface* (API) que faz a ligação entre a aplicação *android*, a base de dados e o *hardware*.

1.1.3 Comunicação

O HTTP é um protocolo da camada de aplicação, do tipo pedido-resposta, que utiliza o TCP como protocolo de transporte O HTTP é o protocolo da camada

de aplicação, do tipo pedido-resposta, que utiliza o padrão protocolo de transporte *Transmission Control Protocol* (TCP), representado em baixo na figura 1.3 [8].

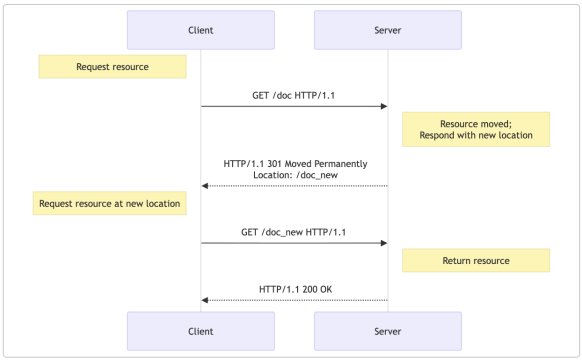


Figura 1.3: Redirecionamento em HTTP [8].

O cliente de uma API é a aplicação que define o conjunto de pedidos que pode efetuar a um servidor e o fluxo de arquitetura: *Representational State Transfer* (REST), que organiza esses pedidos em torno de recursos identificados por *Uniform Resource Locator* (URL). Uma aplicação envia solicitações HTTP do tipo *POST* para uma URL predefinida. Os dados são trocados em formato JSON, um formato textual, independente da linguagem de programação [9].

Na figura que se segue explicita melhor um pedido/resposta simples (*Req*/*Res*: 200) por dados em formato JSON 1.4 [9].

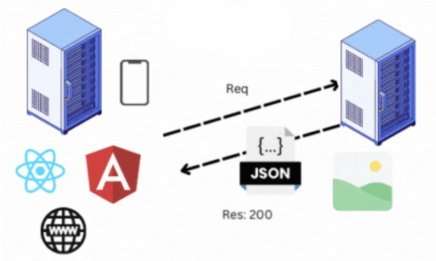


Figura 1.4: Arquitetura de transporte de dados JSON [9].

1.1.4 Base de Dados

O *SQLite* é um Sistema de Gestão de Bases de Dados (SGBD) relacional, de código *open-source*, que implementa a linguagem *Structured Query Language* (SQL) e se destaca por funcionar embebido na própria aplicação, sem necessidade de um servidor de base de dados dedicado nem de configuração prévia [10]. Toda a base de dados é guardada num único ficheiro, o que facilita a sua utilização, cópia e

manutenção, tornando-o adequado a aplicações móveis e a sistemas embebidos com recursos limitados, a figura demonstra o logótipo desta ferramenta 1.5 [10].



Figura 1.5: Logótipo *SQLite database engine* [10].

Neste projeto, o *SQLite* será utilizado para guardar o histórico dos valores medidos pelos sensores e o estado dos atuadores em *real-time*.

1.1.5 Resumo

Em síntese, o *software* designado por AgrIoT é será implementado posteriormente por três camadas principais referidas anteriormente **I:** uma aplicação *android*, desenvolvida em *Kotlin* no *Android Studio* e precedida de *mockups* elaborados no *Figma*, que funciona como interface para o utilizador (subsecção 1.1.1); **II:** um servidor desenvolvido em *Python* com a *framework Flask*, que expõe os serviços necessários através de uma API sobre HTTP (subsecções 1.1.2 e 1.1.3); **III:** uma base de dados *SQLite*, responsável pelo registo dos valores medidos e do estado dos atuadores (subsecção 1.1.4).

A troca de informação entre as diferentes camadas é realizada através de mensagens em formato JSON, e a forma como estes elementos se articulam é detalhada na secção 1.2.

A Tabela 1.1 apresenta o resumo das tecnologias selecionadas, a camada do sistema em que cada uma intervém e a respetiva função.

Tabela 1.1: Resumo das linguagens e tecnologias utilizadas.

Tecnologia	**Camada**	**Função**
Figma	Conceção	*Mockups* e *design* dos *layouts*
Kotlin e *XML*	Aplicação	Lógica e interface da aplicação *android*
Android Studio	Aplicação	Desenvolvimento, teste e compilação da aplicação
Python e *Flask*	Servidor	API e processamento dos dados
HTTP e JSON	Comunicação	Transporte e formato dos dados
SQLite	Dados	Armazenamento persistente

Todo o código-fonte do *software* desenvolvido (aplicação *android*, servidor *Flask* e *scripts* da base de dados), bem como os *mockups* e a documentação associada, encontra-se guardado num repositório público do *GitHub*, o que permite acompanhar a evolução do projeto, manter o histórico de alterações e garantir a reprodutibilidade do trabalho [11].

1.2 Arquitetura de Software

O AgrIoT segue uma arquitetura em camadas, em que cada camada tem uma responsabilidade bem definida. Esta separação permite desenvolver e testar cada parte de forma independente, e alterar uma camada (por exemplo, o aspeto da aplicação) sem afetar as restantes.

1.2.1 Fluxo de Dados

Ciclo de comunicação. A Tabela 1.2 descreve um ciclo típico de comunicação, tomando como exemplo o ligar da ventoinha na aplicação.

Tabela 1.2: Ciclo de comunicação típico

Passo	Fase	Descrição
1	Pedido	A aplicação envia ao servidor um pedido HTTP com mensagem JSON, através da rede privada.
2	Leituras / comandos	Depois de verificar a identidade e o perfil, o servidor envia o comando ao atuador (ou lê o sensor).
3	SQL	O servidor regista na base de dados o novo estado do atuador (ou a leitura obtida).
4	Resposta	O servidor devolve uma resposta HTTP com o resultado e o novo estado, e a aplicação atualiza o ecrã.

Nem todos os ciclos incluem os quatro passos: a consulta do histórico de um sensor, por exemplo, lê apenas a base de dados (passos 1, 2 e 3). Por outro lado, a leitura periódica dos sensores e o respetivo registo ocorrem no servidor sem pedido da aplicação. A aplicação jamais comunica diretamente com o *hardware* nem com a base de dados, todo o acesso passa pelo servidor, que é o ponto central do sistema.

Rede privada (*Tailscale*). A comunicação entre a aplicação e o servidor decorre numa rede privada virtual (*Virtual Private Network* (VPN)) criada com a *Tailscale*. Cada dispositivo (o telemóvel e o servidor) recebe um endereço privado do tipo `100.x.y.z`, e o tráfego entre eles é cifrado. Esta solução permite aceder à estufa a partir de qualquer local sem expor o servidor à Internet pública nem configurar o reencaminhamento de portas no *router*; a aplicação usa o endereço do servidor dentro desta rede nos seus pedidos [12].

1.2.2 Visão Geral da Arquitetura

A Figura 1.6, apresenta as três camadas do sistema de *software*: a aplicação *android* (Camada 1), o servidor *Flask* (Camada 2) e a base de dados *SQLite* (Camada 3).

Os números indicam a ordem de um ciclo típico de comando: (1) pedido, (2) SQL e (3) resposta. *Hardware* (estufa), corresponde à parte física do sistema e inclui os **sensores** (temperatura, humidade do ar, humidade do solo e pressão atmosférica), que medem as grandezas da estufa, e os **atuadores** (luz, ventoinha e aquecedor), que atuam sobre ela. O servidor recebe as leituras dos sensores e envia os comandos aos atuadores (ligação "2 Leituras / comandos" da Figura 1.6).

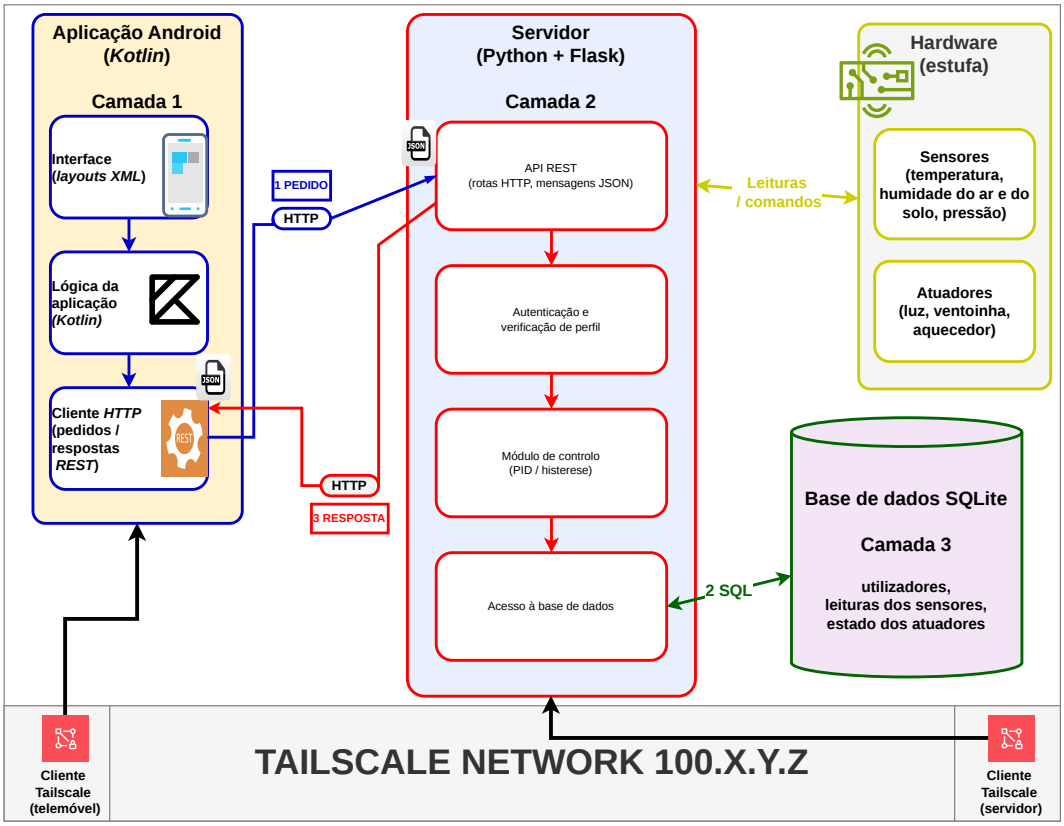


Figura 1.6: Arquitetura de *software* do AgrIoT [13].

Camada 1 – Aplicação *Android*. É a camada de apresentação, executada no telemóvel do utilizador, e é composta por três blocos:

- **Interface (*layouts XML*):** contém os ecrãs desenhados nos *mockups* (secção 1.3). Limita-se a apresentar a informação e a recolher as ações do utilizador, sem conter regras do sistema.
- **Lógica da aplicação (*Kotlin*):** trata os eventos da interface (por exemplo, ligar um atuador), valida os dados introduzidos (campos vazios, palavras-passe diferentes) e converte as respostas do servidor em informação apresentável. Mantém também o perfil da sessão, que determina se as opções de administração são apresentadas (subsecção 1.3.4).

- **Cliente HTTP (pedidos e respostas REST):** constrói os pedidos para o servidor, envia-os em formato JSON e interpreta as respostas recebidas.

Camada 2 – Servidor É a camada de aplicação e o ponto central do sistema. Recebe os pedidos da aplicação, valida-os, executa o controlo da estufa e comunica com o *hardware* e com a base de dados. É composta por quatro blocos:

- **API REST:** é o ponto de entrada e de saída do servidor. Define as rotas para cada recurso (utilizadores, sensores, atuadores e controlador Proporcional-Integral-Derivativo (PID)), associadas aos métodos HTTP *GET*, *POST*, *PUT* e *DELETE*, recebe e devolve mensagens JSON e indica o resultado através dos códigos de estado HTTP.
- **Autenticação e verificação de perfil:** no início de sessão, confirma o ID e a palavra-passe do utilizador. Em cada pedido, verifica a identidade e o perfil associado, e recusa as operações de administração a contas sem esse perfil, tal como descrito na Subsecção 1.3.4.
- **Módulo de controlo (PID / histerese):** lê periodicamente os sensores, compara os valores com os pretendidos (*setpoint*) e decide a atuação. A temperatura é regulada pela ventoinha através de um controlador PID, cujos parâmetros (K_p, K_i, K_d) e *setpoint* podem ser alterados na aplicação; o aquecedor é comandada por histerese.
- **Acesso à base de dados:** concentra todas as operações SQL (guardar leituras, consultar o histórico, gerir utilizadores e registar o estado dos atuadores), isolando o resto do servidor dos detalhes da base de dados.

Camada 3 – Base de dados Guarda de forma persistente a informação do sistema, num único ficheiro gerido pelo servidor, em três conjuntos de dados:

- **Utilizadores:** ID, perfil, estufa associada, estado da conta e palavra-passe. A palavra-passe não é guardada em texto simples, mas sob a forma de *hash*.
- **Leituras dos sensores:** o histórico dos valores medidos, usado, por exemplo, no gráfico das últimas 24 horas.
- **Estado dos atuadores:** o estado atual de cada atuador, que a aplicação consulta para mostrar se está ligado ou desligado.

Apenas o servidor acede à base de dados, o que garante a consistência dos dados e impede que a aplicação contorne as verificações de perfil.

1.3 Conceção da Interface (*Mockups*)

Antes de iniciar a implementação da aplicação *android*, foram desenvolvidos no *Figma* os *mockups* de todos os *layouts* (ecrãs) do AgrIoT. Esta fase permitiu definir a organização da informação, a navegação e a identidade visual da aplicação, de forma a validar a solução antes de escrever código e a reduzir alterações durante o desenvolvimento. Os ecrãs foram desenhados com as dimensões de um telemóvel (412×915 píxeis), em orientação vertical.

1.3.1 Visão Geral dos Ecrãs

A Figura 1.7, apresenta o conjunto dos nove *layouts* desenhados [4]. Os cinco primeiros, na primeira linha, correspondem ao utilizador comum: iniciar sessão, página inicial, sensores, atuadores e controlador PID, implementando assim os métodos do HTTP do tipo *GET, POST, PUT*. Os quatro restantes, na segunda linha, pertencem ao perfil de administrador responsáveis pela gestão de utilizadores, implementando assim os métodos do HTTP do tipo *GET, POST, PUT, DELETE*.

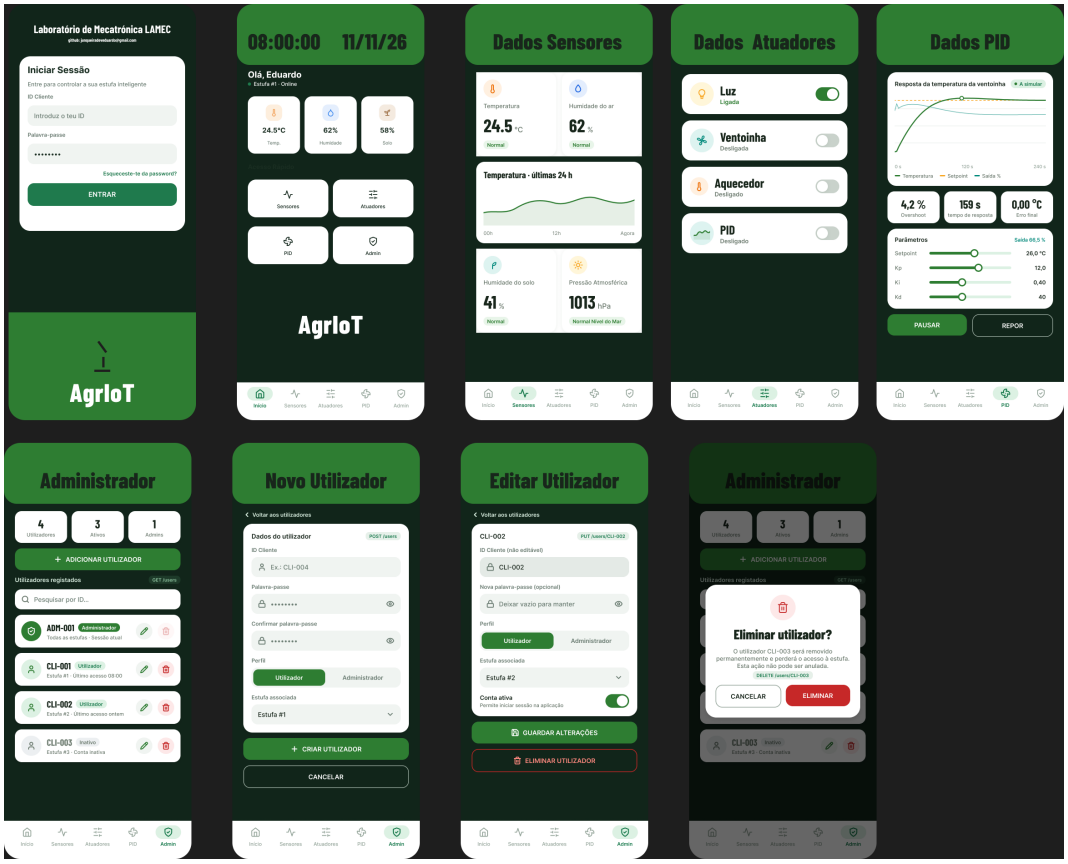


Figura 1.7: Conjunto dos *mockups* da aplicação *android* AgrIoT [4].

1.3.2 *Design layout*

A interface segue uma linguagem visual única em todos os ecrãs, associada ao âmbito do projeto referenciado em cima na Figura 1.7. Os elementos que a definem estão resumidos na Tabela 1.3.

Tabela 1.3: Linguagem visual da interface da aplicação.

Elemento	**Descrição**
Cores	Fundo escuro, verde mais claro para a barra superior e para as ações principais, e cartões brancos que destacam a informação. O vermelho é usado apenas em ações de *delete*, como eliminar um utilizador.
Tipografia	*Barlow Condensed* nos títulos e nos valores numéricos, para os destacar, e *Inter* nos restantes textos, pela legibilidade em ecrãs pequenos.
Barra superior	Faixa verde com cantos inferiores arredondados, que mostra o título do ecrã (ou a hora e a data, na página inicial).
Barra de navegação inferior	Estática e comum aos cinco ecrãs principais (Início, Sensores, Atuadores, *PID* e *Admin*), com o separador ativo destacado em verde.
Cartões, *chips* e interruptores	Os cartões agrupam a informação, os *chips* indicam estados (por exemplo, "Normal" ou "A simular") e os interruptores ligam e desligam os atuadores.

A reutilização destes componentes garante coerência entre *layouts* e facilita a implementação posterior, já que cada elemento passa a corresponder a um componente *XML* reutilizável.

1.3.3 Utilizador

Iniciar Sessão. Ecrã de *login*, com o nome da Unidade Curricular (UC) de Laboratório de Mecatrónica (LAMEC), os campos "ID Cliente" e "Palavra-passe", a ligação "Esqueceste-te da password?" e o botão "ENTRAR". A parte inferior mostra o logótipo principal AgrIoT.

Página Inicial. Após o *login*, esta página mostra a hora e a data, a saudação ao utilizador e o estado da estufa ("Online"). Apresenta ainda um resumo rápido com a temperatura, a humidade do ar e a humidade do solo, e um bloco de acesso rápido às áreas Sensores, Atuadores, PID e *Admin*, que se encontram na barra de navegação inferior.

Sensores. Quatro cartões mostram os valores atuais de todos os sensores: de temperatura, humidade do ar, humidade do solo e pressão atmosférica, cada um com um *chip* de estado. No centro, um gráfico mostra a evolução da temperatura nas últimas 24 horas.

Atuadores. Lista todos atuadores: luz, ventoinha, aquecedor e PID com o estado "Ligada" ou "Desligada" e um interruptor para os comandar.

PID. Mostra uma possível simulação com um gráfico da resposta da temperatura da ventoinha, com o *setpoint* e a saída em percentagem. Apresenta ainda as métricas de desempenho (*overshoot*, tempo de resposta e erro final) e permite ajustar com precisão os valores de *setpoint* e os ganhos K_p, K_i e K_d. Os botões "PAUSAR" e "REPOR" controlam esta simulação.

1.3.4 Perfil de Administrador

A aplicação distingue dois perfis de utilizador, definidos no momento do início de sessão: **utilizador** e **administrador**. O perfil é associado a cada conta no momento da sua criação e não pode ser escolhido pelo próprio utilizador durante o *login*.

O acesso à área de administração é restrito ao perfil de administrador. Quando a sessão é iniciada com uma conta de utilizador, o separador *Admin* da barra de navegação inferior e o respetivo atalho na página inicial são apresentados na mesma, mas a gestão de utilizadores não está acessível ao clique. Quando a sessão é iniciada com uma conta de administrador, ambos ficam visíveis e acessíveis ao clique. Este bloqueio de clique evita que utilizadores comuns criem, alterem ou eliminem contas, e garante que apenas um responsável controla esta aplicação num todo. A Tabela 1.4 relaciona cada *layout* com a operação HTTP correspondente do administrador.

Tabela 1.4: *Layouts* do administrador e operação HTTP associada.

Ecrã	**Operação**	**Função**
Administrador (lista)	*GET*	Consultar os utilizadores existentes, com pesquisa por ID
Novo Utilizador	*POST*	Criar utilizador (ID, palavra-passe, perfil e estufa)
Editar Utilizador	*PUT*	Alterar palavra-passe, perfil, estufa e estado da conta
Eliminar Utilizador	*DELETE*	Remover utilizador, com confirmação prévia

GET. Apresenta o total de utilizadores, os ativos e os administradores, o botão "ADICIONAR UTILIZADOR" e uma pesquisa por ID. Cada utilizador surge num cartão com ID, perfil, estufa associada e estado, e com botões de editar e eliminar. A

conta do administrador em sessão não pode ser eliminada, para evitar que o sistema fique sem administrador.

POST, PUT. Os dois pedidos são semelhantes, o que facilita a implementação. Na edição, o ID aparece bloqueado, porque identifica o utilizador, e a nova palavra-passe é opcional. O perfil escolhe-se num seletor (Utilizador ou Administrador) e a conta pode ser ativada ou desativada com um interruptor.

DELETE. Por ser uma ação irreversível, o pedido é confirmado numa janela sobre a lista, que escurece o fundo e usa o vermelho para destacar o botão de eliminação.

1.4 Trabalho Futuro

Implementação e integração.

- Implementar a aplicação *android* em *Kotlin*, a partir dos *mockups* e dos *layouts XML* já definidos.
- Desenvolver o servidor *Flask*, com a API REST, a autenticação por perfil e o módulo de controlo (PID e histerese).
- Criar a base de dados *SQLite* e ligar o servidor ao *hardware* da estufa (sensores e atuadores).
- Configurar a rede privada com a *Tailscale* e validar o ciclo de comunicação completo, da aplicação ao atuador.

Validação e evolução.

- Executar os procedimentos e vetores de teste previstos e afinar, na estufa real, os ganhos e o *setpoint* do controlador PID.
- Acrescentar notificações e alertas para valores fora dos limites definidos.
- Alargar o histórico de leituras (por exemplo, a períodos superiores a 24 horas) e permitir a gestão de várias estufas por utilizador.
- Reforçar a segurança, nomeadamente a recuperação de palavra-passe, indicada no ecrã de início de sessão.

Referências

[1] Android. *Kotlin and Android | Android Developers*. Nov. de 2026. URL: https://developer.android.com/kotlin (Citado nas páginas 1, 2).

[2] Jetbrains. *Kotlin for Android | Kotlin Documentation*. Nov. de 2026. URL: https://kotlinlang.org/docs/android-overview.html (Citado na página 1).

[3] Android. *Download Android Studio & App Tools - Android Developers*. Nov. de 2026. URL: https://developer.android.com/studio (Citado na página 2).

[4] Figma - Eduardo Junqueira. *LAMEC-AgrIoT – Figma*. Nov. de 2026. URL: https://www.figma.com/design/nGE7FGaUWJ6amzYHQ8rDRO/LAMEC-AgrIoT?node-id=0-1&p=f&t=PisOiK9KkCRR1XlV-0 (Citado nas páginas 2, 8).

[5] Python Software Foundation Legal Statements. *Welcome to Python.org*. Nov. de 2026. URL: https://www.python.org/ (Citado na página 2).

[6] AWS. *O que é Python? – Explicação sobre a linguagem Python – AWS*. Nov. de 2026. URL: https://aws.amazon.com/pt/what-is/python/ (Citado na página 2).

[7] Werkzeug WSGI toolkit. *Welcome to Flask — Flask Documentation*. Nov. de 2026. URL: https://flask.palletsprojects.com/en/stable/ (Citado na página 2).

[8] Mozilla. *Uma visão geral do HTTP - HTTP | MDN*. Nov. de 2026. URL: https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Guides/Overview (Citado na página 3).

[9] Medium - Klaus Dellano. *As 6 Arquiteturas de API mais procuradas | by Klaus Dellano | HavanLabs | Medium*. Nov. de 2026. URL: https://medium.com/havanlabs/as-6-arquiteturas-de-api-mais-procuradas-25128fdbcd72 (Citado na página 3).

[10] SQLite Consortium members. *SQLite Home Page*. Nov. de 2026. URL: https://www.sqlite.org/ (Citado nas páginas 3, 4).

[11] GitHub - Eduardo Junqueira. *JunqueiraDevEduardo/AgrIoT: Aplicação Mobile LAMEC - Estufa Inteligente*. Nov. de 2026. URL: https://github.com/JunqueiraDevEduardo/AgrIoT (Citado na página 4).

[12] Tailscale - Eduardo Junqueira. *Machines - Tailscale*. Nov. de 2026. URL: `https://console.tailscale.com/admin/machines` (Citado na página 5).

[13] Draw.io - Eduardo Junqueira. *Arquitetura de Software*. Nov. de 2026 (Citado na página 6).